Übungsblatt 7: CPU-Architekturen

Mikrocomputertechnik 3 · 32 Orientierungspunkte · ca. 60–75 Minuten

Aufgabe 1: Eine sechsstufige Pipeline (6 Punkte)

Stufen F, D1, D2, EX, MEM, WB, jeweils ein Takt; maximal ein Start je Takt. Fenster: erster Start bis letztes WB.

1. Sieben unabhängige Befehle: Fensterlänge, CPI, IPC? Warum ist IPC nicht eins? (2 P)
2. Der zweite Befehl liest das Ergebnis des ersten in D2. Kein Forwarding, Ergebnis von I1 in WB sechs, Schreiben vor Lesen im selben Takt. Ein wartender Leser hält alle jüngeren Befehle auf. Wann beginnt I2 mit EX? Bestimmen Sie neues Fenster und CPI. (2 P)
3. Zwei Übersetzungen derselben Aufgabe: 160 Befehle bei CPI 1,2 beziehungsweise 240 bei CPI 0,9, beide 160 MHz. Berechnen und vergleichen Sie Laufzeiten. (2 P)

Ihre Lösung:

Aufgabe 2: Andere Register und Operationen (6 Punkte)

RV32I-Modell: s0 adressiert ein gültiges ausgerichtetes Wort mit Wert 18; t4 ist sechs, t5 vierzehn, t6 neun, a1 zwei.

```
lw  t2, 0(s0)     # A
add t3, t2, t4    # B
xor t2, t5, t6    # C
sub a0, t2, a1    # D
```

1. Nennen Sie alle RAW-Kanten zwischen den vier Befehlen. (2 P)
2. Nennen Sie WAR und WAW. Warum sind diese Namenskonflikte? (1 P)
3. Anfangsabbildung: s0 P8, t2 P2, t4 P4, t5 P5, t6 P6, a1 P9. Neue Ziele P20, P21, P22, P23 in Programmreihenfolge. Erstellen Sie eine Tabelle physischer Quellen und Ziele. Welche t2-Version liest B? (2 P)
4. Bestimmen Sie endgültig t2, t3 und a0. (1 P)

Ihre Lösung:

Aufgabe 3: Ein längerer Load hält Bestätigung auf (6 Punkte)

A–D aus Aufgabe 2 sind vor Takt eins dekodiert und umbenannt. Ein Issue je Takt, eine ALU und eine Load/Store-Einheit. A startet in eins und wird Ende sechs fertig; andere Befehle Ende ihres Starttakts. Nutzer starten frühestens im Folgetakt des Produzenten. Zwei Retires je Takt, strikt in Programmreihenfolge, frühestens nach Fertigstellung. In-Order wählt nur den nächsten Programmplatz; Out-of-Order wählt den ältesten bereiten Kandidaten.

1. Bestimmen Sie Start, Fertigstellung und Retire für In-Order. (2 P)
2. Bestimmen Sie diese Zeiten für Out-of-Order. (2 P)
3. Vergleichen Sie Fensterlänge, IPC, CPI, Zeitreduktion und Speedup. Ist der Load dadurch früher fertig? (2 P)

Ihre Lösung:

Aufgabe 4: Ein anfangs nicht genommener Sprung (5 Punkte)

2-Bit-Zähler startet `01`; `00/01` sagen NT, `10/11` T vorher. T erhöht, NT senkt um eins, saturierend von null bis drei.

1. Verfolgen Sie NT, T, T, T, NT, NT, NT, T: alter Zustand, Vorhersage, Ist, neuer Zustand. Zählen Sie Treffer. (3 P)
2. Separates additives Modell: Basis-CPI 0,9, Sprunganteil 15 Prozent, Fehlerquote innerhalb der Sprünge zwölf Prozent, zehn Extratakte je Fehler. Keine Strafüberlappung. Bestimmen Sie Gesamt-CPI, Zeit- und Durchsatzänderung bei gleicher Arbeit und Frequenz. (2 P)

Ihre Lösung:

Aufgabe 5: Ein anderer bestätigter Ausgangszustand (5 Punkte)

Vor A aus Aufgabe 2: t2 ist 31, t3 40, a0 55. Ein Vorgänger hat s1 bereits mit 27 bestätigt.

1. A meldet einen synchronen Ladefehler; C und D sind intern schon fertig. Welche genannten Architekturwerte bleiben erhalten? Was darf nicht bestätigt werden? (2 P)
2. Separat liefert A erfolgreich 18. Ein Interrupt wird genau nach Bestätigung von A, aber vor B angenommen. Welche vier Werte sieht der Handler? (1 P)
3. Ein falsch vorhergesagter Bereichspfad wird verworfen. Welche wertabhängige Kopplung wäre für einen Cache-Seitenkanal nötig? Beweist eine einzelne kurze Zugriffszeit den geheimen Wert? (2 P)

Ihre Lösung:

Aufgabe 6: Drei statt zwei Summenketten (4 Punkte)

Der gültige Puffer enthält $[9, 1, 8, 2, 7, 3, 6]$, n ist sieben.

```
#include <stddef.h>
#include <stdint.h>
uint64_t sum3(const uint32_t *a, size_t n) {
    uint64_t s0 = 0, s1 = 0, s2 = 0;
    size_t groups = n / 3;
    for (size_t j = 0; j < groups; ++j) {
        s0 += a[3*j]; s1 += a[3*j+1]; s2 += a[3*j+2];
    }
    for (size_t i = 3*groups; i < n; ++i) s0 += a[i];
    return s0 + s1 + s2;
}
```

1. Bestimmen Sie Teilsummen vor Restbehandlung und Rückgabe. (1 P)
2. Begründen Sie die Indexgrenzen und das Verhalten bei n null mit Nullzeiger. (1 P)
3. Welche Unabhängigkeit entsteht gegenüber einer einzigen Summenkette? (1 P)
4. Beweist Ergebnisgleichheit oder eine O3-Übersetzung einen Laufzeitgewinn? (1 P)

Ihre Lösung:

Optional: am Rechner prüfen

Ergänzen Sie `src/lab07_ilp.c` um `sum3` aus Aufgabe 6 und neue Tests: $[9, 1, 8, 2, 7, 3, 6]$ ergibt 36; $[4, 10]$ ergibt 14; leere Eingabe ergibt null. Vergleichen Sie Funktion und erzeugten Assembler bei O1 und O3 getrennt von Laufzeitmessungen.